

Manual Técnico do Script `bttng_integration.py` – Sistema Hórus

Versão do Documento: 1.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python 3 + PostgreSQL + API BTTNG (via biblioteca `btt`)

Este manual técnico descreve em detalhes o script `bttng_integration.py`, que atua como o módulo de integração com a API BTTNG no sistema Hórus. O script é responsável por conectar-se à API, buscar credenciais vazadas das últimas 24 horas (ou período configurado), validar e preparar os dados para inserção no banco de dados, e registrar o processo em logs de auditoria. Ele garante sincronização diária automática, focando em credenciais relevantes para domínios militares/governamentais, e integra com o processamento do Hórus para deduplicação e armazenamento.

O documento explica as funcionalidades principais, configurações, funções disponíveis, restrições de uso e mecanismos de integração implementados. Ele é destinado a administradores, desenvolvedores e mantenedores que precisam compreender ou modificar o comportamento da sincronização com fontes externas de threat intelligence.

1. Introdução e Propósito

O `bttng_integration.py` é o componente dedicado à integração com a API BTTNG (da Apura) no Hórus, uma plataforma de Threat Intelligence para detecção e gestão de credenciais vazadas. Ele busca dados novos (das últimas 24 horas por default), aplica filtros de tempo, valida credenciais (usuário, senha, site), prepara estruturas para inserção e orquestra a sincronização com o banco PostgreSQL via funções externas. O script opera de forma autônoma via execução direta ou integrado a schedulers (ex: CRON para diário), com logging extensivo para monitoramento e auditoria.

A classe principal, `BTTNGIntegration`, encapsula a lógica de API, utilizando a biblioteca `btt` para consultas, `psycopg2` para inserções no banco e `dotenv` para credenciais seguras. Ele enfatiza em buscas eficientes com filtros, validação rigorosa e relatórios de resultados (totais encontrados/validados/inseridos), auxiliando na automação de alertas via RT.

1.1 Dependências e Integrações

- **Linguagem:** Python 3.
- **Banco de Dados:** PostgreSQL (via psycopg2), com conexões contextuais e inserções em batch.
- **Bibliotecas:** logging (logs estruturados), datetime e timedelta (manipulação de datas), typing (anotações), contextlib (context managers), psycopg2 (DB ops), btt (APIClient e SearchFilter para BTTNG), os e dotenv (variáveis de ambiente), logging_config (logger customizado).
- **Integrações Externas:** API BTTNG (endpoint e key via .env), banco de dados repo (tabelas credenciais/geral e audit_log), credenciais_processor.py (validação e inserção).
- **Scripts Relacionados:** Chamado por app.py (sincronização manual), create_rt_ticket_daily.py (alertas pós-sync), credenciais_processor.py (validações compartilhadas), scheduler_bttnng.py (agendamento).

2. Funcionalidades Principais

O `bttnng_integration.py` fornece uma integração robusta com a API BTTNG, com foco em automação e validação. Abaixo, as principais funcionalidades:

- **Conexão à API:** Estabelece e fecha conexões seguras com BTTNG usando APIClient.
- **Filtros de Busca:** Cria filtros de tempo (últimas 24h) via SearchFilter, com logging de datas UTC.
- **Busca de Credenciais:** Consulta API por credenciais novas (* query, order by timestamp desc), retorna iterador para processamento eficiente.
- **Contagem de Resultados:** Conta totais encontrados sem carregar todos os dados.
- **Validação e Preparação:** Valida usuário/site (via funções externas), gera hashes SHA1, filtra rejeições (ex: usuário inválido, site não permitido).
- **Inserção no Banco:** Prepara lista de dicts e chama inserir_no_banco para batch insertion com deduplicação.
- **Auditoria e Logging:** Registra resultados em audit_log (totais encontrados/validados/etc.) e logs detalhados (progresso, erros).
- **Sincronização Diária:** Método completo para run (conecta, busca, valida, insere, audita), com handling de exceções.
- **Execução Direta:** Bloco main para teste/sync manual, carregando .env.

3. Configurações

O script define configurações globais e de ambiente via dotenv e os.getenv:

- **Credenciais API:** BTTNG_ENDPOINT (ex: 'https://bttng.apura.com.br/'), BTTNG_API_KEY, BTTNG_COMPANY_ID (opcional, filtra por empresa).
- **Logger:** Usa default_logger de logging_config para info/error com exc_info.
- **Datas:** UTC para from_date/to_date (timedelta days=1 default).
- **Banco de Dados:** Usa get_active_db_config()['repo'] para conexões; autocommit true em auditoria.
- **Outras:** Query fixa '*' (todos), order_by ["@timestamp:desc"], remove_password=False (mantém senhas), source_bttng=True para rastreo.

Restrições: Depende de .env (falha se ausente); company_id opcional (usa env se não passado); erros de tabela audit_log warning apenas (não falha sync).

4. Funções e Métodos Principais

O script define uma função auxiliar e uma classe BTTNGIntegration com métodos modulares. Cada um é descrito abaixo, incluindo parâmetros, funcionalidades e restrições:

- **safe_getattr(obj, attr, default=''):** Acessa atributos de LeakedData com try/except, retorna default em falhas.
- **init(endpoint, api_key, company_id=None):** Inicializa client como None, logger, carrega company_id de env se ausente.
- **connect():** Cria APIClient, loga sucesso. Retorna bool. Restrição: Raise não capturado em erros.
- **disconnect():** Fecha client se existente, loga.
- **build_time_range_filter(days=1):** Cria SearchFilter para indexed_at (UTC now - days to now). Loga datas. Retorna filter.
- **search_new_leaks(limit=None, days=1):** Cria filter, consulta API com query '*', filters e order_by. Retorna iterador de LeakedData. Retorna vazio em erros.
- **count_leaks(days=1):** Usa search_new_leaks para contar itens no iterador (log a cada 5000). Retorna int (0 em erros).
- **validate_and_prepare_credentials(credentials: Iterator):** Itera LeakedData, valida user/site, gera hashes SHA1, filtra rejeições. Retorna listas de válidas/rejeitadas (dicts prontos para insert).
- **insert_credentials_to_horus(valid_credentials: List[Dict]):** Chama inserir_no_banco com source_bttnng=True. Retorna dict novas/duplicadas/falhas (0 em erros).
- **run_daily_sync(days=1):** Pipeline completo: conecta, conta, busca, valida, insere, audita. Retorna dict status/mensagem/totais. Finally: desconecta.
- **log_sync_to_audit(resultado: Dict):** Insere em audit_log com mensagem resumida. Warning em UndefinedTable ou DB errors (não raise).
- **if name == 'main':** Execução direta: carrega .env, init classe, run_daily_sync(days=1), loga resumo.

5. Restrições e Segurança

- **Período de Busca:** Default 24h (days=1); sem suporte a períodos fracionários. Filtro em indexed_at (não data do vazamento).
- **Validações:** Reusa validar_usuario/parse_site (rígidas para domínios); rejeita se user/site inválidos ou senha vazia. Hashes SHA1 para deduplicação.
- **Performance:** Iterador para grandes volumes (não carrega tudo em RAM); contagem loga progressivamente. Restrição: Sem paralelismo; API pode limitar rates.
- **Segurança:** API key via .env (não hardcoded); conexões com try/except amplo. Logs sem senhas claras; source_bttnng para rastreio. Raise propagado em erros críticos.
- **Erros:** Handled com logs error/exc_info; retorna vazio/0/dict erro em falhas. Auditoria skip se tabela ausente.
- **Restrições Gerais:** Query fixa '*' (sem customização); depende de btt lib (assuma instalada). Sem reconexão automática; exec direta falha sem .env.

Nota Final

O bttnng_integration.py é essencial para automação de threat intel, mas alterações devem preservar filtros e logging para conformidade (ex: LGPD para credenciais). Consulte o código para extensões ou adicione filtros avançados via btt. Em produção, integre com CRON e monitore logs para falhas de API/conexão.